Свёрточные нейронные сети

&&

Харитон Пицик

2026-04-01

Клуб машинного обучения

Recap: Функции активации

&&

Ранее мы уже рассматривали некоторые функции активации. Их основное предназначение – научить модель выявлять **нелинейные** признаки.

Recap: Функции активации

&&

Ранее мы уже рассматривали некоторые функции активации. Их основное предназначение – научить модель выявлять **нелинейные** признаки.

Ранее мы остановились на $ReLU$ и $LeakyReLU$, которые достаточно хорошо себя показывают.

Также мы остановились на том, что для задачи бинарной классификации досаточно использовать сигмоиду:

$$\sigma(x) = \frac{1}{1 + e^{-x}}$$

Однако остался вопрос, как решать задачу мультиклассовой классификации?

Softmax

Для решения задачи мультиклассовой классификации применяется следующая функция активации:

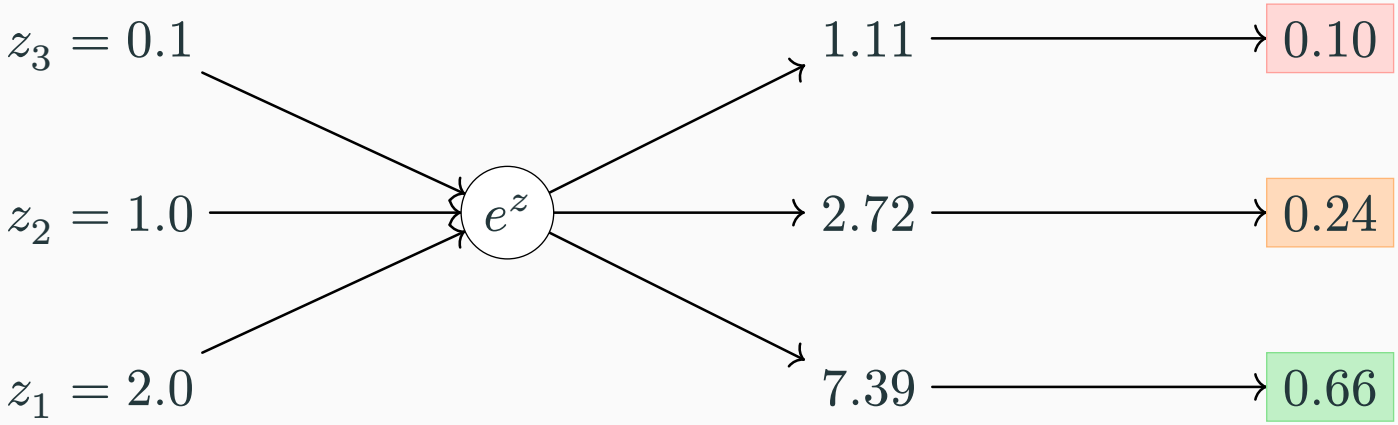
$$f(z_i) = \frac{e^{z_i}}{\sum(e^{z_i})}$$

Для решения задачи мультиклассовой классификации применяется следующая функция активации:

$$f(z_i) = \frac{e^{z_i}}{\sum(e^{z_i})}$$

Такая функция активации называется **Softmax**, её идея заключается в том, чтобы отображать каждый элемент в $[0, 1]$.

$$\sum p_i = 1.0$$



Выходит, что мы получаем вектор из n элементов, сумма которого всегда 1.

Выходит, что мы получаем вектор из n элементов, сумма которого всегда 1.

В дальнейшем эти выходы можно интерпретировать как вероятность каждого из классов.

Обработка фото. Почему MLP плох?

Почему MLP плох?

&&

Представьте, что вы хотите научить НС находить что-то на фотографии. Вы берёте картинку размером 1024×1024 пикселя. Каждый пиксель – это три числа. Итого на вход вашей нейросети поступает вектор длины

Почему MLP плох?

&&

Представьте, что вы хотите научить НС находить что-то на фотографии. Вы берёте картинку размером 1024×1024 пикселя. Каждый пиксель – это три числа. Итого на вход вашей нейросети поступает вектор длины

$$D = 1024 \times 1024 = 1,048,576$$

Далее давайте предположим, что у нас есть какое-то скрытое состояние размерности 4096. В таком случае количество весов только на этом слое составит:

Далее давайте предположим, что у нас есть какое-то скрытое состояние размерности 4096. В таком случае количество весов только на этом слое составит:

$$|W| = 4096 \times 1,048,576$$

Далее давайте предположим, что у нас есть какое-то скрытое состояние размерности 4096. В таком случае количество весов только на этом слое составит:

$$|W| = 4096 \times 1,048,576$$

Держите в голове, что это только один слой: обычно таких слоёв сильно больше, следовательно, количество весов становится просто непозволительным.

Почему MLP плох?

&&

Давайте также учитывать, что если у модели слишком много параметров относительно данных, то она достаточно быстро переобучится, и врядли какая-то регуляризация может здесь помочь.

Почему MLP плох?

&&

Давайте также учитывать, что если у модели слишком много параметров относительно данных, то она достаточно быстро переобучится, и врядли какая-то регуляризация может здесь помочь.

Непонятно, откуда столько данных брать, а даже если они и появятся, то непонятно, как эту модель обучать.

Почему MLP плох? &&

Однако проблема не только в количестве параметров.

Почему MLP плох?

&&

Однако проблема не только в количестве параметров.

Полносвязный слой обладает свойством, которое для изображений является существенным: он **игнорирует пространственную структуру**.

Давайте мы предположим, что у нас есть 2 одинаковые картинки, но одна из них сдвинута на n пикселей вправо.

Почему MLP плох? &&

Однако проблема не только в количестве параметров.

Полносвязный слой обладает свойством, которое для изображений является существенным: он **игнорирует пространственную структуру**.

Давайте мы предположим, что у нас есть 2 одинаковые картинки, но одна из них сдвинута на n пикселей вправо.

Формально эти картинки по прежнему идентичные и обладают одинаковым содержимым, но MLP будет распознавать их как разные, поскольку опять же, каждый пиксель обрабатывается независимо.

Значит нам нужно решить 2 проблемы:

Что делать? &&

Значит нам нужно решить 2 проблемы:

1. Независимость пикселей в контексте модели;

Что делать? &&

Значит нам нужно решить 2 проблемы:

1. Независимость пикселей в контексте модели;
2. Невосприимчивость к повторяющимся паттернам.

Идея состоит в том, чтобы обрабатывать изображение **локально**: вместо того, чтобы соединять каждый пиксель с каждым нейроном, мы будем смотреть на небольшие окрестности пикселей – **патчи**.

Идея состоит в том, чтобы обрабатывать изображение **локально**: вместо того, чтобы соединять каждый пиксель с каждым нейроном, мы будем смотреть на небольшие окрестности пикселей – **патчи**.

Такой подход сразу решает обе проблемы:

Идея состоит в том, чтобы обрабатывать изображение **локально**: вместо того, чтобы соединять каждый пиксель с каждым нейроном, мы будем смотреть на небольшие окрестности пикселей – **патчи**.

Такой подход сразу решает обе проблемы:

- Соседние пиксели обрабатываются **вместе**, а не независимо;

Идея состоит в том, чтобы обрабатывать изображение **локально**: вместо того, чтобы соединять каждый пиксель с каждым нейроном, мы будем смотреть на небольшие окрестности пикселей – **патчи**.

Такой подход сразу решает обе проблемы:

- Соседние пиксели обрабатываются **вместе**, а не независимо;
- Один и тот же фильтр применяется **по всему изображению**, а значит паттерн будет найден вне зависимости от его положения.

Свёртка

&&

Ключевая операция в CNN – это **свёртка** (convolution). Возьмём фильтр размером $k \times k$ и будем “скользить” им по изображению, на каждом шаге вычисляя скалярное произведение фильтра и патча изображения под ним:

Ключевая операция в CNN – это **свёртка** (convolution). Возьмём фильтр размером $k \times k$ и будем “скользить” им по изображению, на каждом шаге вычисляя скалярное произведение фильтра и патча изображения под ним:

$$(I * K)(i, j) = \sum_{m=0}^{k-1} \sum_{n=0}^{k-1} I_{i+m, j+n} \cdot K(m, n)$$

где I – входное изображение, K – ядро свёртки.

Ключевая операция в CNN – это **свёртка** (convolution). Возьмём фильтр размером $k \times k$ и будем “скользить” им по изображению, на каждом шаге вычисляя скалярное произведение фильтра и патча изображения под ним:

$$(I * K)(i, j) = \sum_{m=0}^{k-1} \sum_{n=0}^{k-1} I_{i+m, j+n} \cdot K(m, n)$$

где I – входное изображение, K – ядро свёртки.

Размер выходного изображения (feature map) при входе $H \times W$, ядре $k \times k$ и шаге (stride) s:

Ключевая операция в CNN – это **свёртка** (convolution). Возьмём фильтр размером $k \times k$ и будем “скользить” им по изображению, на каждом шаге вычисляя скалярное произведение фильтра и патча изображения под ним:

$$(I * K)(i, j) = \sum_{m=0}^{k-1} \sum_{n=0}^{k-1} I_{i+m, j+n} \cdot K(m, n)$$

где I – входное изображение, K – ядро свёртки.

Размер выходного изображения (feature map) при входе $H \times W$, ядре $k \times k$ и шаге (stride) s:

$$H_{\text{out}} = \left\lfloor \frac{H - k}{s} \right\rfloor + 1, \quad W_{\text{out}} = \left\lfloor \frac{W - k}{s} \right\rfloor + 1$$

Padding

&&

Заметьте, что после свёртки изображение уменьшается. Чтобы сохранить пространственные размеры, применяют **padding** – дополнение изображения нулями по краям.

Padding

&&

Заметьте, что после свёртки изображение уменьшается. Чтобы сохранить пространственные размеры, применяют **padding** – дополнение изображения нулями по краям.

При $\text{padding} = p$ формула меняется:

Padding

&&

Заметьте, что после свёртки изображение уменьшается. Чтобы сохранить пространственные размеры, применяют **padding** – дополнение изображения нулями по краям.

При $\text{padding} = p$ формула меняется:

$$H_{\text{out}} = \left\lfloor \frac{H - k + 2p}{s} \right\rfloor + 1$$

При $k = 3, s = 1, p = 1$ размер изображения сохраняется: $H_{\text{out}} = H$.

Свойство разделяемых весов

&&

Ключевое отличие свёрточного слоя от полносвязного – **разделяемые веса**.

Свойство разделяемых весов

&&

Ключевое отличие свёрточного слоя от полносвязного – **разделяемые веса**.

Один фильтр размером $k \times k$ применяется ко **всем** позициям изображения. Количество параметров одного фильтра:

$$|K| = k \times k \times C_{\text{in}}$$

где C_{in} – число каналов входа. Для $k = 3$, $C_{\text{in}} = 3$: всего 27 параметров против MLP.

Свойство разделяемых весов

&&

Ключевое отличие свёрточного слоя от полносвязного – **разделяемые веса**.

Один фильтр размером $k \times k$ применяется ко **всем** позициям изображения. Количество параметров одного фильтра:

$$|K| = k \times k \times C_{\text{in}}$$

где C_{in} – число каналов входа. Для $k = 3$, $C_{\text{in}} = 3$: всего 27 параметров против MLP.

Если мы используем C_{out} фильтров, то суммарно:

$$|W_{\text{conv}}| = k \times k \times C_{\text{in}} \times C_{\text{out}}$$

Pooling

&&

После свёртки обычно применяют **пулинг** – операцию, которая уменьшает пространственный размер feature map и делает представление более устойчивым к малым сдвигам.

После свёртки обычно применяют **пулинг** – операцию, которая уменьшает пространственный размер feature map и делает представление более устойчивым к малым сдвигам.

Наиболее популярный вариант – **Max Pooling**: берём максимум по окну $p \times p$:

$$y(i, j) = \max_{0 \le m, n < p} x(i \cdot s + m,, j \cdot s + n)$$

Это даёт два эффекта:

После свёртки обычно применяют **пулинг** – операцию, которая уменьшает пространственный размер feature map и делает представление более устойчивым к малым сдвигам.

Наиболее популярный вариант – **Max Pooling**: берём максимум по окну $p \times p$:

$$y(i, j) = \max_{0 \leq m, n < p} x(i \cdot s + m,, j \cdot s + n)$$

Это даёт два эффекта:

- **Сжатие**: размер карты уменьшается в s раз по каждому измерению;

После свёртки обычно применяют **пулинг** – операцию, которая уменьшает пространственный размер feature map и делает представление более устойчивым к малым сдвигам.

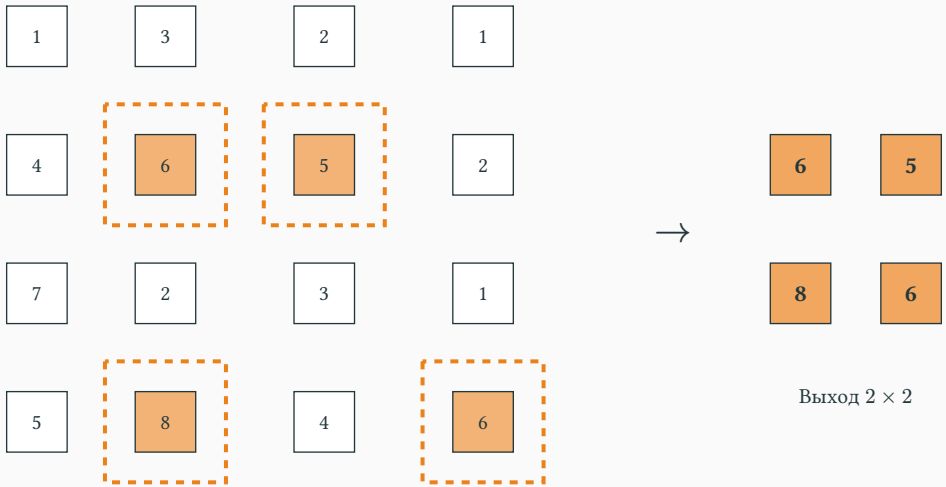
Наиболее популярный вариант – **Max Pooling**: берём максимум по окну $p \times p$:

$$y(i, j) = \max_{0 \le m, n < p} x(i \cdot s + m,, j \cdot s + n)$$

Это даёт два эффекта:

- **Сжатие**: размер карты уменьшается в s раз по каждому измерению;
- **Инвариантность**: небольшой сдвиг паттерна внутри окна не меняет результат.

Pooling



Вход 4 × 4, окно 2 × 2

Архитектура CNN

&&

Свёрточная нейронная сеть обычно строится следующим образом:

Свёрточная нейронная сеть обычно строится следующим образом:

$$\underbrace{[\text{Conv} \to \text{ReLU}] \times N}_{\text{feature extraction}} \to \text{Flatten} \to \underbrace{[\text{Linear} \to \text{ReLU}] \times M}_{\text{classification}}$$

Архитектура CNN

&&

Свёрточная нейронная сеть обычно строится следующим образом:

$$\underbrace{[\text{Conv} \to \text{ReLU}] \times N}_{\text{feature extraction}} \to \text{Flatten} \to \underbrace{[\text{Linear} \to \text{ReLU}] \times M}_{\text{classification}}$$

- **Conv + ReLU** – извлекают локальные признаки, постепенно увеличивая глубину (C_{out}) и уменьшая пространственный размер;

Свёрточная нейронная сеть обычно строится следующим образом:

$$\underbrace{[\text{Conv} \to \text{ReLU}] \times N}_{\text{feature extraction}} \to \text{Flatten} \to \underbrace{[\text{Linear} \to \text{ReLU}] \times M}_{\text{classification}}$$

- **Conv + ReLU** – извлекают локальные признаки, постепенно увеличивая глубину (C_{out}) и уменьшая пространственный размер;
- **Flatten** – разворачивает трёхмерный тензор $H \times W \times C$ в вектор;

Свёрточная нейронная сеть обычно строится следующим образом:

$$\underbrace{[\text{Conv} \to \text{ReLU}] \times N}_{\text{feature extraction}} \to \text{Flatten} \to \underbrace{[\text{Linear} \to \text{ReLU}] \times M}_{\text{classification}}$$

- **Conv + ReLU** – извлекают локальные признаки, постепенно увеличивая глубину (C_{out}) и уменьшая пространственный размер;
- **Flatten** – разворачивает трёхмерный тензор $H \times W \times C$ в вектор;
- **Linear** – классический MLP-классификатор поверх извлечённых признаков.

Для входа 1024×1024 и первого свёрточного слоя с 64 фильтрами 3×3:

Для входа 1024×1024 и первого свёрточного слоя с 64 фильтрами 3×3:

$$|W_{\text{conv}}| = 3 \times 3 \times 64 = 1728$$

Для входа 1024×1024 и первого свёрточного слоя с 64 фильтрами 3×3:

$$|W_{\text{conv}}| = 3 \times 3 \times 64 = 1728$$

В то же время MLP:

$$|W_{\text{MLP}}| = 1024 \times 1024 \times 4096 \approx 1.29 \times 10^{10}$$

Недостатки CNN

&&

Недостатки CNN

&&

1. Инвариантность к сдвигу;

Недостатки CNN

&&

1. Инвариантность к сдвигу;
2. Потеря данных при пуллинге;

Недостатки CNN

&&

1. Инвариантность к сдвигу;
2. Потеря данных при пуллинге;
3. Требовательность к данным.

Многоканальная свёртка

Цветные изображения

&&

До сих пор мы рассматривали одноканальные (чёрно-белые) изображения. Но реальная фотография задаётся слегка сложнее:

$$H \times W \times C_{\text{in}}$$

где $C_{\text{in}} = 3$ для RGB-изображения.

Цветные изображения

&&

До сих пор мы рассматривали одноканальные (чёрно-белые) изображения. Но реальная фотография задаётся слегка сложнее:

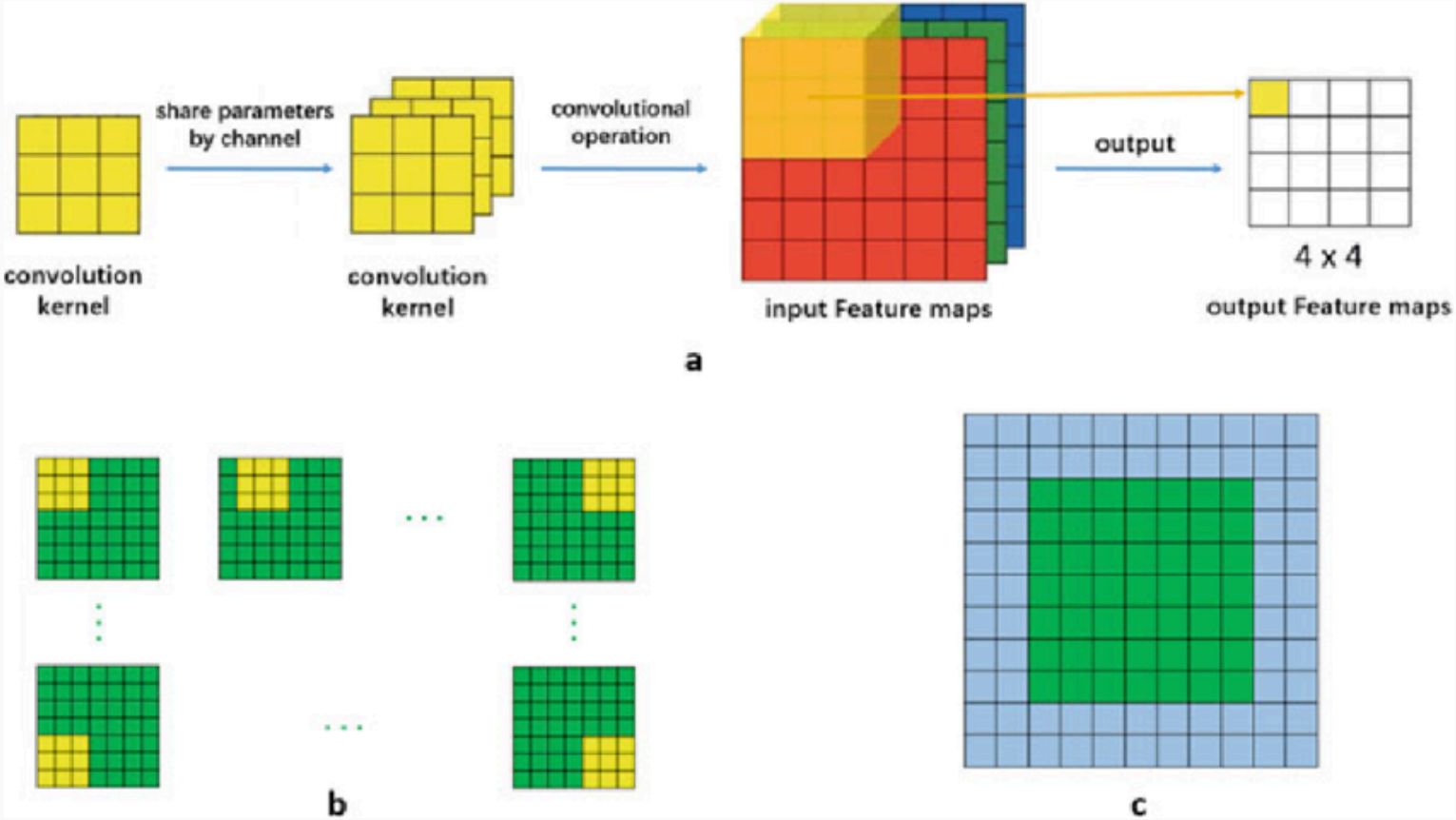
$$H \times W \times C_{\text{in}}$$

где $C_{\text{in}} = 3$ для RGB-изображения.

Фильтр тоже становится трёхмерным: $k \times k \times C_{\text{in}}$. Каждый фильтр “смотрит” сразу на все каналы в своей окрестности и выдаёт **одно число**.

Цветные изображения

&&



Несколько фильтров

&&

Один фильтр ищет один паттерн. Но нам нужно много признаков – значит, используем C_{out} фильтров.

Несколько фильтров

&&

Один фильтр ищет один паттерн. Но нам нужно много признаков – значит, используем C_{out} фильтров.

Каждый фильтр K_c размером $k \times k \times C_{\text{in}}$ применяется к входному тензору и даёт одну карту признаков (feature map) размером $H_{\text{out}} \times W_{\text{out}}$.

Несколько фильтров

&&

Один фильтр ищет один паттерн. Но нам нужно много признаков – значит, используем C_{out} фильтров.

Каждый фильтр K_c размером $k \times k \times C_{\text{in}}$ применяется к входному тензору и даёт одну карту признаков (feature map) размером $H_{\text{out}} \times W_{\text{out}}$.

Итого, выход свёрточного слоя – тензор:

$$H_{\text{out}} \times W_{\text{out}} \times C_{\text{out}}$$

Число параметров слоя:

$$|W| = k \times k \times C_{\text{in}} \times C_{\text{out}} + C_{\text{out}}$$

где последнее слагаемое – смещение на каждый выходной канал.

Что учат фильтры?

&&

Интуиция: фильтры в разных слоях учатся находить разные вещи.

Что учат фильтры? &&

Интуиция: фильтры в разных слоях учатся находить разные вещи.

- **Первые слои** – простые паттерны: края, градиенты, цветовые переходы;

Что учат фильтры?

&&

Интуиция: фильтры в разных слоях учатся находить разные вещи.

- **Первые слои** – простые паттерны: края, градиенты, цветовые переходы;
- **Средние слои** – более сложные структуры: текстуры, части объектов;

Что учат фильтры?

&&

Интуиция: фильтры в разных слоях учатся находить разные вещи.

- **Первые слои** – простые паттерны: края, градиенты, цветовые переходы;
- **Средние слои** – более сложные структуры: текстуры, части объектов;
- **Глубокие слои** – высокоуровневые признаки: “колесо”, “глаз”, “лицо”.

Что учат фильтры?

&&

Интуиция: фильтры в разных слоях учатся находить разные вещи.

- **Первые слои** – простые паттерны: края, градиенты, цветовые переходы;
- **Средние слои** – более сложные структуры: текстуры, части объектов;
- **Глубокие слои** – высокоуровневые признаки: “колесо”, “глаз”, “лицо”.

Это происходит автоматически, в процессе обучения. Сеть сама решает, какие фильтры ей нужны.

Другие виды пулинга

Другие виды пулинга

&&

Мы уже видели Max Pooling, который уменьшает feature map в 2 раза.

Другие виды пулинга

&&

Мы уже видели Max Pooling, который уменьшает feature map в 2 раза.

Но есть другой популярный вариант – **Average Pooling**: берём не максимум, а среднее по окну:

$$y(i, j) = \frac{1}{p^2} \sum_{0 \le m, n < p} x(i \cdot s + m, j \cdot s + n)$$

Global Average Pooling (GAP) &&

А что если взять среднее не по маленькому окну, а **по всей карте признаков** целиком?

Global Average Pooling (GAP) &&

А что если взять среднее не по маленькому окну, а **по всей карте признаков** целиком?

Это называется **Global Average Pooling**. Он превращает тензор $H \times W \times C$ в вектор длины C:

$$\mathrm{GAP}(X)_c = \frac{1}{H \cdot W} \sum_{i,j} X_{i,j,c}$$

Global Average Pooling (GAP) &&

А что если взять среднее не по маленькому окну, а **по всей карте признаков** целиком?

Это называется **Global Average Pooling**. Он превращает тензор $H \times W \times C$ в вектор длины C:

$$\mathrm{GAP}(X)_c = \frac{1}{H \cdot W} \sum_{i,j} X_{i,j,c}$$

Это радикально сокращает количество параметров: вместо `Flatten` $\longrightarrow$ большого `Linear` слоя – просто усреднение по пространству, и сразу классификатор.

Global Average Pooling (GAP) &&

А что если взять среднее не по маленькому окну, а **по всей карте признаков** целиком?

Это называется **Global Average Pooling**. Он превращает тензор $H \times W \times C$ в вектор длины C:

$$\mathrm{GAP}(X)_c = \frac{1}{H \cdot W} \sum_{i,j} X_{i,j,c}$$

Это радикально сокращает количество параметров: вместо `Flatten` $\longrightarrow$ большого `Linear` слоя – просто усреднение по пространству, и сразу классификатор.

Именно GAP используется в большинстве современных архитектур вместо старой схемы Flatten $\longrightarrow$ Dense.

Итоговая картина

&&

Полный пайплайн обучения CNN выглядит так:

Итоговая картина

&&

Полный пайплайн обучения CNN выглядит так:

1. Прямой проход: $X \to [\text{Conv} \to \text{ReLU}] \to \text{Pooling} \to \text{Flatten} \to \text{Linear} \to \text{Softmax} \to \hat{p}$

Итоговая картина

&&

Полный пайплайн обучения CNN выглядит так:

1. Прямой проход: $X \to [\text{Conv} \to \text{ReLU}] \to \text{Pooling} \to \text{Flatten} \to \text{Linear} \to \text{Softmax} \to \hat{p}$
2. Считаем потерю: $\mathcal{L} = -\log \hat{p}_{\text{correct}}$

Итоговая картина

&&

Полный пайплайн обучения CNN выглядит так:

1. Прямой проход: $X \to [\text{Conv} \to \text{ReLU}] \to \text{Pooling} \to \text{Flatten} \to \text{Linear} \to \text{Softmax} \to \hat{p}$
2. Считаем потерю: $\mathcal{L} = -\log \hat{p}_{\text{correct}}$
3. Обратный проход: backpropagation считает градиенты по всем весам

Итоговая картина

&&

Полный пайплайн обучения CNN выглядит так:

1. Прямой проход: $X \to [\text{Conv} \to \text{ReLU}] \to \text{Pooling} \to \text{Flatten} \to \text{Linear} \to \text{Softmax} \to \hat{p}$
2. Считаем потерю: $\mathcal{L} = -\log \hat{p}_{\text{correct}}$
3. Обратный проход: backpropagation считает градиенты по всем весам
4. Обновляем веса: градиентный спуск

Softmax + кросс энтропия идут в паре – это стандартная комбинация для задач классификации.